IFRN - TECNICAS DE ANALISE DE DADOS

ATIVIDADE 3 - AGRUPAMENTO DE DADOS

Robério Batista de Azevêdo Júnior e Daniel Goes | insurance.csv

Objetivo e metodologia

Aplicar K-Means e Agrupamento Hierarquico (Ward) aos dados preparados.
O pre-processamento foi entregue na Atividade 2 e e reutilizado pelo codigo.
A coluna charges possui valores numericos continuos (nao e uma classe).
Para avaliar os clusters, charges e discretizada por quantis em tres faixas:
Baixo, Medio e Alto. A coluna charges NAO participa da formacao dos clusters.
O numero de clusters e K = 3, para comparar com as tres faixas de custo.

Resultados da dupla - valores a confirmar na execucao

Metrica	K-Means	Hierarquico
ARI	0.204379	0.136107
NMI	0.213334	0.133954
Coeficiente de Silhueta	0.185624	0.129670

Matrizes de contingencia

K-Means:	Cluster 0	Cluster 1	Cluster 2
Baixo	261	17	79
Medio	11	211	134
Alto	110	135	111

Hierarquico:	Cluster 0	Cluster 1	Cluster 2
Baixo	66	60	231
Medio	200	134	22
Alto	94	114	148

Consideracoes finais

K-Means teve desempenho relativo superior nas tres metricas apresentadas.
Entretanto, os valores baixos de Silhueta, ARI e NMI sugerem separacao
e correspondencia limitadas entre clusters e faixas de custo.
As faixas foram CRIADAS por quantis; nao existiam como rotulos no CSV.

Fonte: Kaggle - Medical Cost Personal Datasets

ATIVIDADE 3 - CODIGO PYTHON COMPLETO

Importa o pre-processamento definido na Atividade 2.

```python
"""IFRN - Atividade 3: Agrupamento do Medical Cost Personal Datasets.
Dupla: Robério Batista de Azevêdo Júnior e Daniel Goes.
O código de pré-processamento consta da entrega da Atividade 2.
"""

from pathlib import Path
import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import (
    adjusted_rand_score, normalized_mutual_info_score, silhouette_score,
)
from atividade_02_preprocessamento import preparar_dados

RAIZ = Path(__file__).resolve().parents[1]
RESULTADOS = RAIZ / "resultados"


def main():
    # 1. Reutilizar o pré-processamento implementado na Atividade 2.
    X_treino_preparado, _, y_treino, _ = preparar_dados()

    # 2. Discretizar charges em faixas de custo SOMENTE para avaliação.
    # As classes Baixo, Médio e Alto não existem originalmente no CSV.
    # charges não participa das variáveis usadas para obter clusters.
    y_treino_faixas = pd.qcut(
        y_treino, q=3, labels=["Baixo", "Médio", "Alto"]
    )

    # 3. Aplicar dois algoritmos de agrupamento com três clusters.
    kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
    labels_kmeans = kmeans.fit_predict(X_treino_preparado)
    hierarquico = AgglomerativeClustering(n_clusters=3, linkage="ward")
    labels_hierarquico = hierarquico.fit_predict(X_treino_preparado)

    # 4. Métricas externas e interna.
    ari_kmeans = adjusted_rand_score(y_treino_faixas, labels_kmeans)
    nmi_kmeans = normalized_mutual_info_score(y_treino_faixas, labels_kmeans)
    sil_kmeans = silhouette_score(X_treino_preparado, labels_kmeans)
    ari_hier = adjusted_rand_score(y_treino_faixas, labels_hierarquico)
    nmi_hier = normalized_mutual_info_score(y_treino_faixas, labels_hierarquico)
    sil_hier = silhouette_score(X_treino_preparado, labels_hierarquico)

    resultados = pd.DataFrame({
        "Métrica": ["ARI", "NMI", "Coeficiente de Silhueta"],
        "K-Means": [ari_kmeans, nmi_kmeans, sil_kmeans],
        "Agrupamento Hierárquico": [ari_hier, nmi_hier, sil_hier],
    })
    print("\nComparativo dos algoritmos:")
    print(resultados.to_string(index=False, float_format=lambda n: f"{n:.6f}"))

    # 5. Tabelas cruzadas: clusters x faixas de charges.
```

ATIVIDADE 3 - CODIGO PYTHON COMPLETO

Importa o pre-processamento definido na Atividade 2.

```
    tabela_kmeans = pd.crosstab(
        y_treino_faixas, labels_kmeans,
        rownames=["Faixa de charges"], colnames=["Cluster K-Means"],
    )
    tabela_hier = pd.crosstab(
        y_treino_faixas, labels_hierarquico,
        rownames=["Faixa de charges"], colnames=["Cluster Hierárquico"],
    )
    print("\nDistribuição: K-Means por faixa de custo\n", tabela_kmeans)
    print("\nDistribuição: Hierárquico por faixa de custo\n", tabela_hier)

    # 6. Salvar arquivos de resultado em uma única pasta.
    RESULTADOS.mkdir(parents=True, exist_ok=True)
    resultados.to_csv(RESULTADOS / "atividade3_metricas.csv", index=False)
    tabela_kmeans.to_csv(RESULTADOS / "atividade3_matriz_kmeans.csv")
    tabela_hier.to_csv(RESULTADOS / "atividade3_matriz_hierarquico.csv")


if __name__ == "__main__":
    main()
```